

Intro to Amazon ECS

by Abby Fuller, AWS



Amazon EC2 Container Service (ECS)

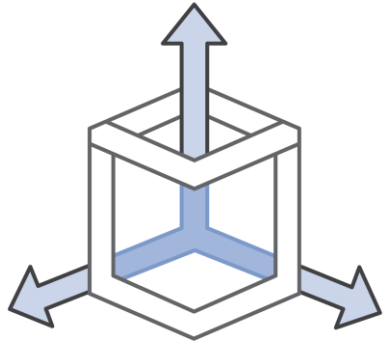
Highly scalable, high performance container management system.

Eliminates the need to install, operate, and scale your own container management infrastructure.

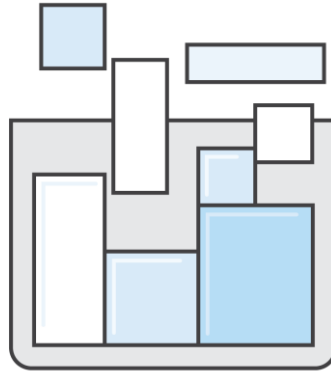


Amazon EC2 Container Service (ECS)

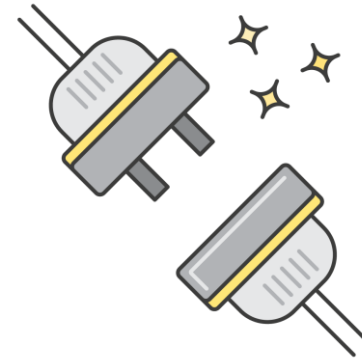
ECS provides a managed platform for:



Cluster
management

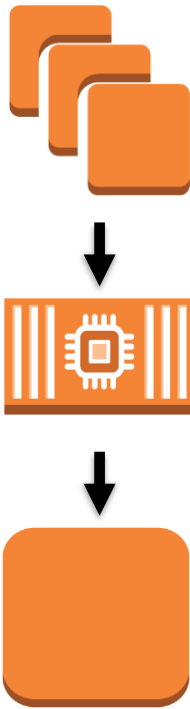


Container
orchestration



Deep AWS
integration

How does ECS map to traditional workloads?

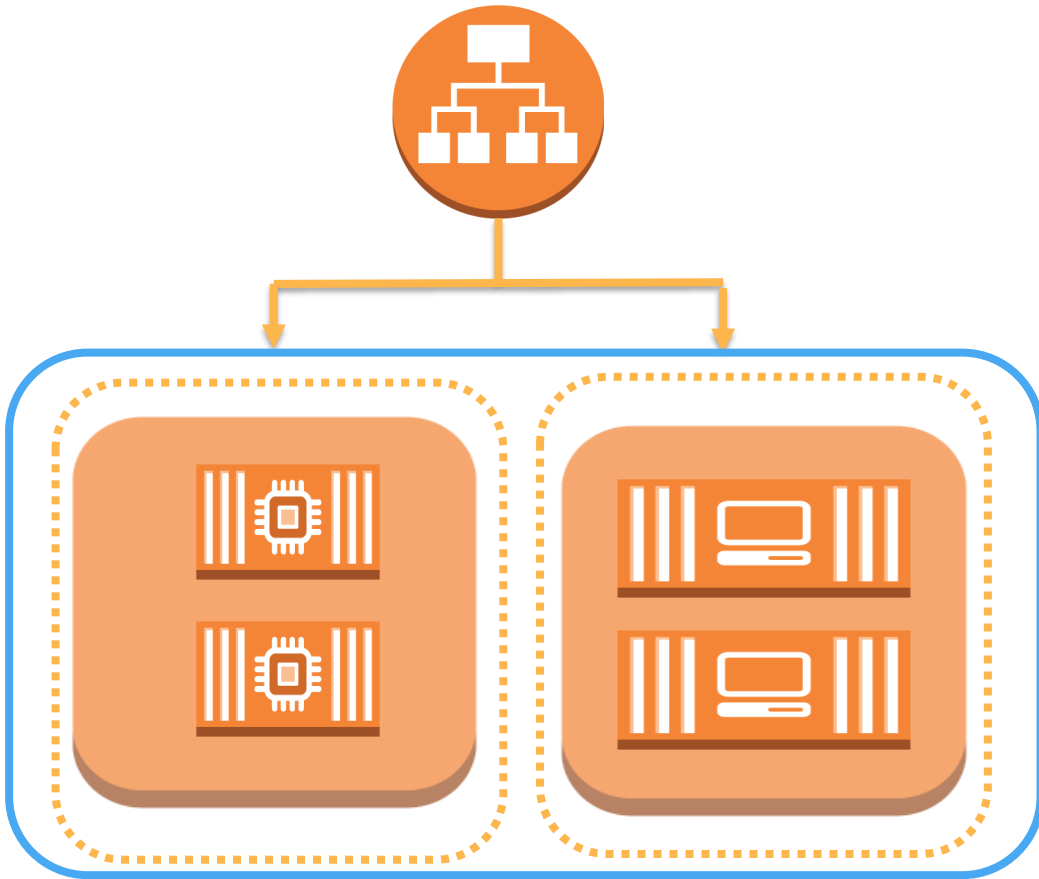


Instances: standard EC2 boxes. Once registered to a Cluster, your Tasks run here

Services: layer that manages and places Tasks

Tasks: container wrapper and configuration around processes running on the instance

How does ECS work?



Load balancer: (ALB or EC2 classic) routes traffic to the cluster instances.

Cluster is made up of one or more EC2 instances

Each **cluster instance** runs one or more **Services**

How does ECS work?



Each **cluster instance** runs one or more **Services**

A **Service** controls things like the number of copies of a Task you want running (Desired Count), and registers your Service with a load balancer

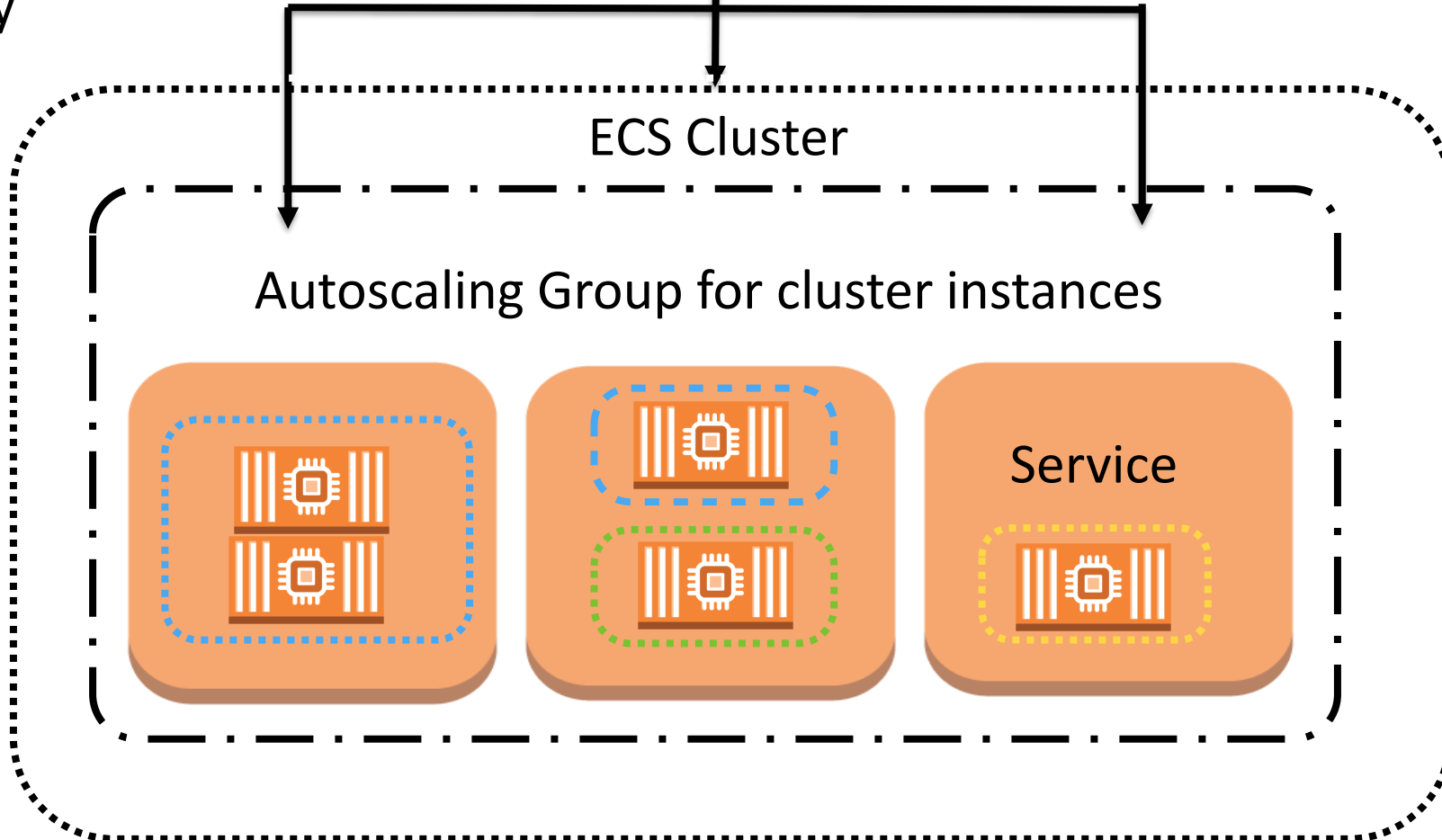
A **Task Definition** controls things like container image, environment variables, resource allocation, logger, and other parameters



ECR Registry



Application Load Balancer



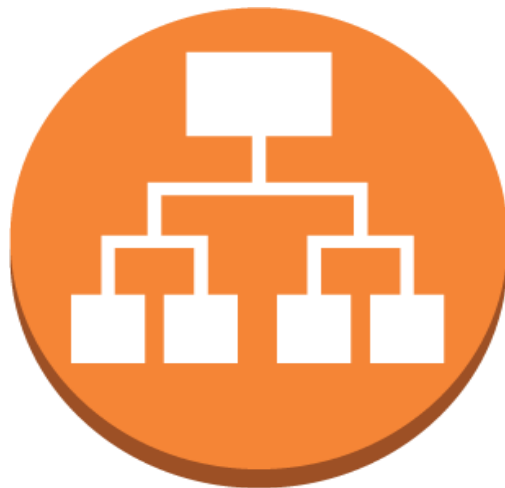
ECS Cluster

Autoscaling Group for cluster instances

Service

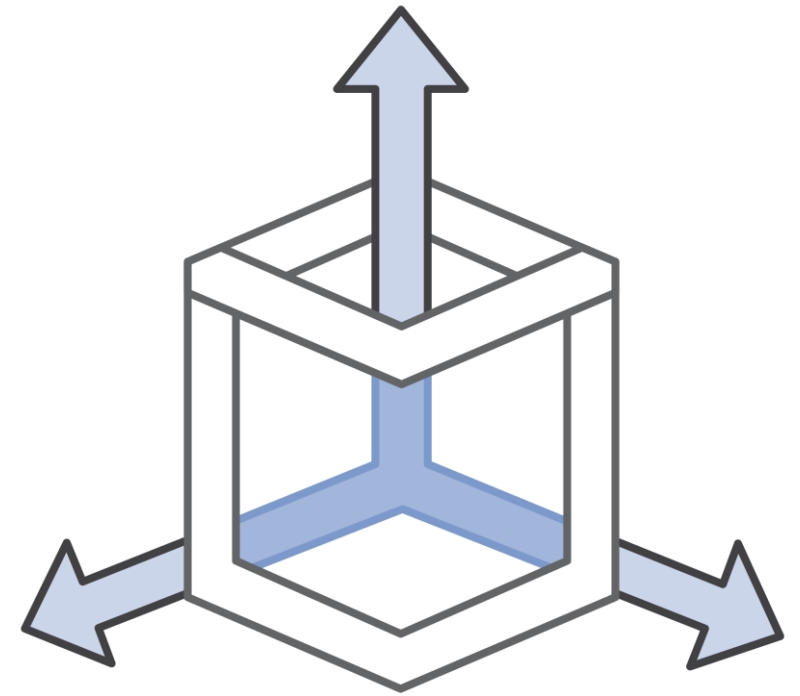
Let's talk about ALB

- **Define routing rules based on content.** Fancy way of saying “send traffic to different services based on endpoint”. This is magical.
- As a bonus, this allows ECS to allocate ports dynamically rather than statically, and one ALB can handle multiple services.



Why ECS?

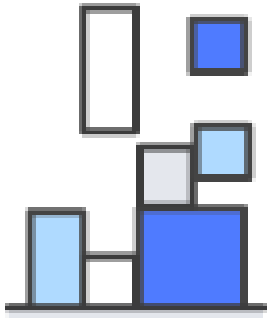
Bottom line: containers and microservices can require a lot of orchestration and moving pieces. ECS removes a lot of this heavy lifting.



Let's get (feature) specific



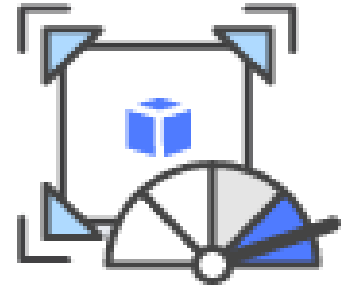
A few features, but many more.



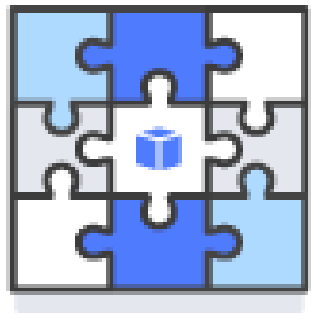
Amazon ECS Task Placement



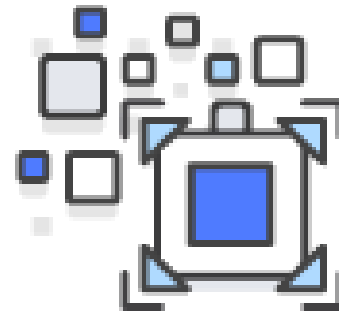
IAM Roles for Tasks



Flexible scaling for performance



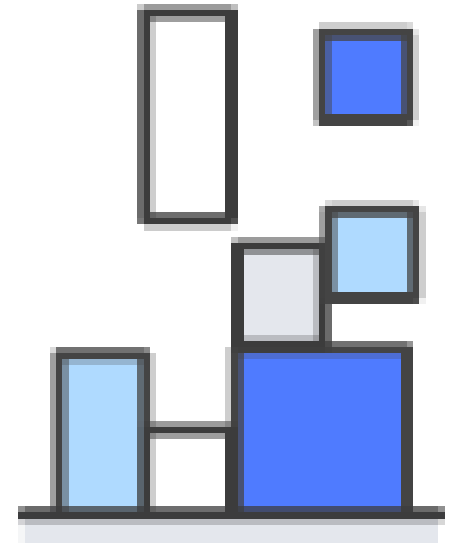
Amazon ECS Event Stream for Cloudwatch Logs



Fast, hassle-free deployments

Amazon ECS Task Placement

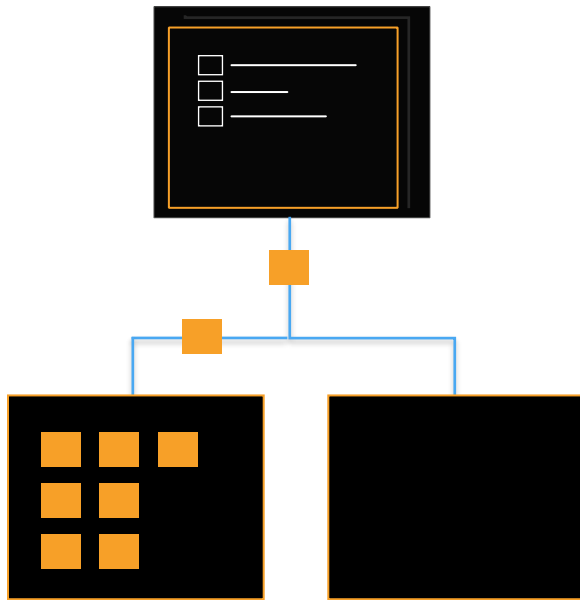
- A *task placement strategy* is an algorithm for selecting instances for task placement, or tasks for termination
- A *task placement constraint* is a rule taken into consideration during task placement
- Strategies and constraints can be used together



How can strategies and policies be used?

Name	Example
AMI ID	<code>attribute:ecs.ami-id == ami-eca289fb</code>
Availability Zone	<code>attribute:ecs.availability-zone == us-east-1a</code>
Instance Type	<code>attribute:ecs.instance-type == t2.small</code>
Distinct Instances	<code>type="distinctInstances"</code>
Custom	<code>attribute:stack == prod</code>

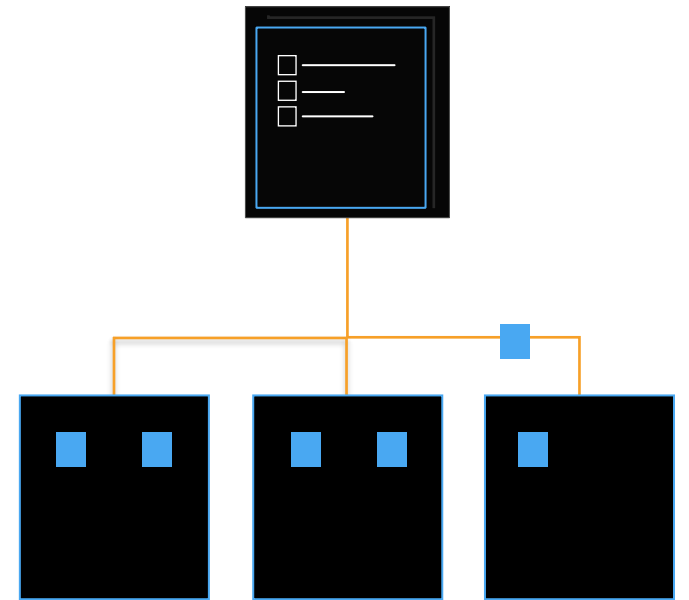
Multiple strategies are supported



Binpacking

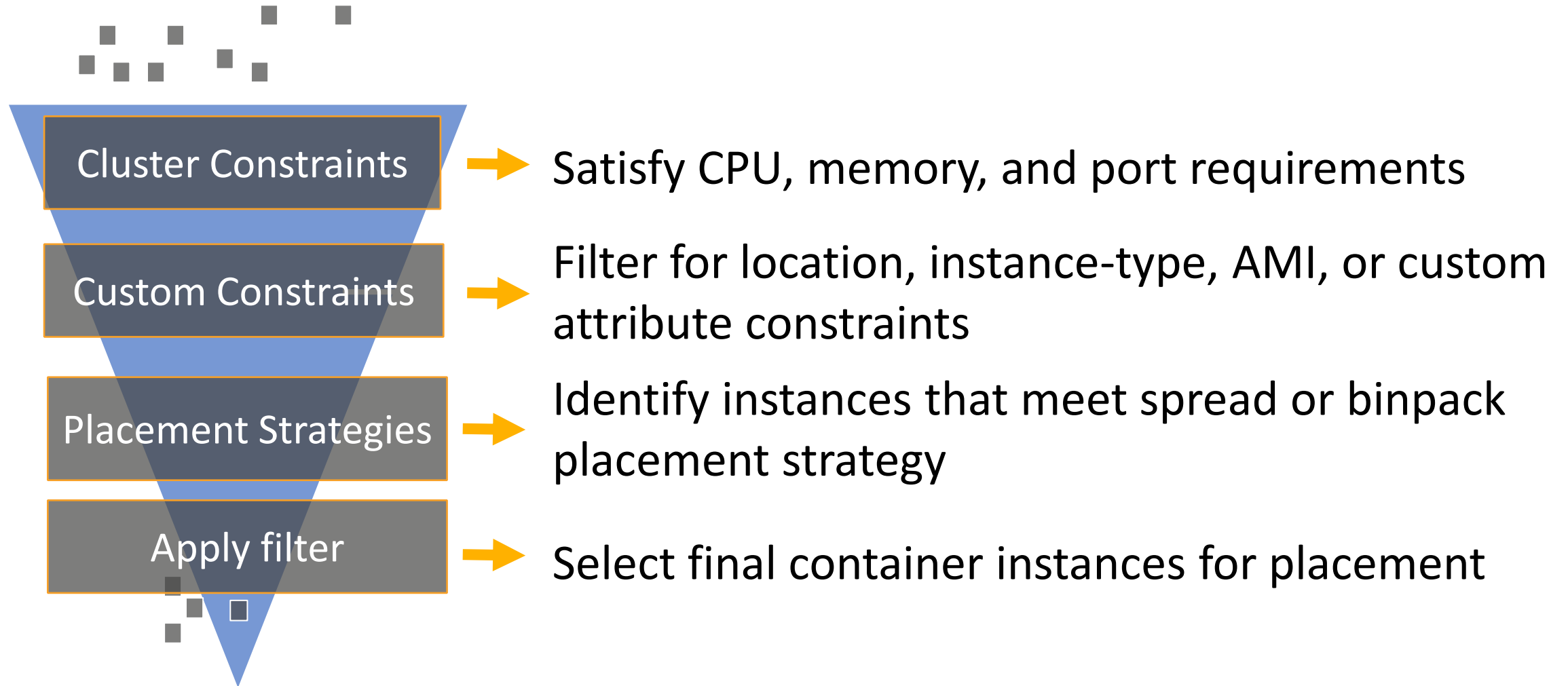


Random



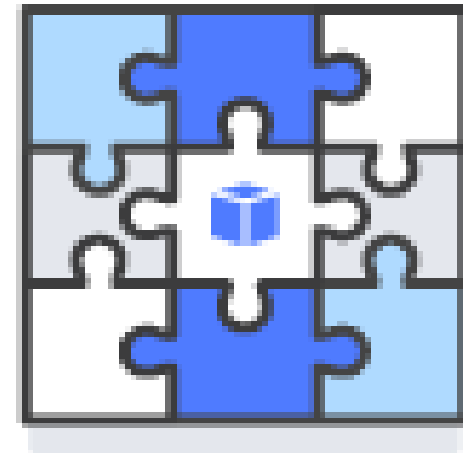
Spread

How it works



Amazon ECS Event Stream for Cloudwatch Logs

- Receive near real-time updates about both the current state of the container instances within the ECS Cluster, and the current state of all tasks running on those container instances.
- Can be used to build custom schedulers, or to monitor cluster state and handle those state changes by consuming events with other AWS services, such as Lambda.



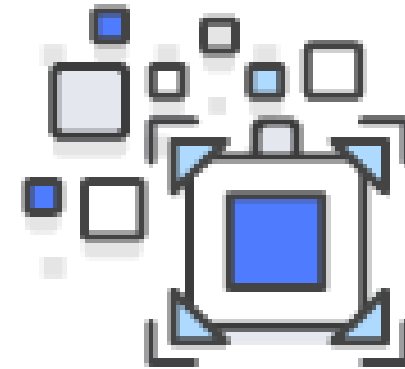
IAM Roles for ECS Tasks

- **Specify an IAM role used by the containers in a task.**
- **Credential Isolation:** containers can only access the role for the specific task that they are assigned to.
- **Authorization:** Unauthorized containers cannot access IAM role credentials defined for other tasks.
- **Auditability:** Audit through CloudTrail. Can track the Task credentials taskARN to show which task is using which role.



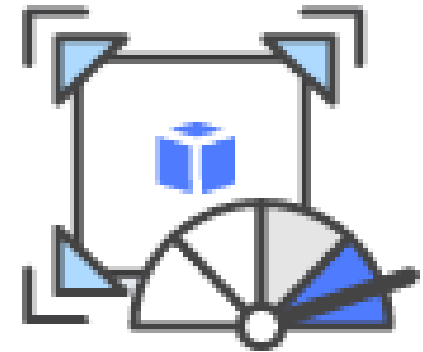
Fast, hassle-free deployments

- **Services deploy and scale quickly.** Very easily extensible through API calls; for example, trigger a deployment based on a commit to a branch on Github through your CI tool.
- **Plus, extra protection baked in.** ECS will only drain connections from the previous Task Definition if the new Task Definition passes health checks.



Flexible scaling for performance

- **Scale a service up or down based on CloudWatch alarms.** Autoscaling is built into the Service during the registration process.
- Since Clusters are part of EC2 Autoscaling Groups, you can also **scale the Cluster itself based on resources**, like you would any other group.



A great disturbance in the force

- With the shift to microservices, comes a shift in thinking: more and more options are moving from just the server level to the containers themselves.
- Don't just move a service over to containers and call it a day: decompose and rebuild.
- Security (IAM), scaling (Task-level autoscaling), traffic distribution (ALB and NLB), configuration, settings → all happening at the container/service level now.

With more services comes more responsibility

- More moving pieces
- Safety and security first
- Choose the right option (tool, language, setting) that works for you.
- Use your resources! Document, alert, automate.

Some ECS resources

- AWS docs: <https://aws.amazon.com/ecs/>
- ECS first run wizard: <https://console.aws.amazon.com/ecs/home?region=us-east-1>
- Nathan Peck's ECS repo: <https://github.com/nathanpeck/awesome-ecs>
- More talks of mine: <https://aws.amazon.com/evangelists/abby-fuller/>
- ECS "Getting Started" workshop: <https://www.github.com/abby-fuller/ecs-demo>